

ECS401: Unit 8 Lab Progress Test

FEEDBACK: Answers and Explanations

Work hard and you will eventually be able to program. It is really important that you understand why the answers here are what they are. If anything is confusing then get someone to explain.

1. Complete the dry run table when this code is executed. What is printed?

```
int score1 = 57;           // L1
int score2 = 21;           // L2
score2 = score1;           // L3
score2 = 49;               // L4
System.out.println(score1 + ":" + score2); // L5
```

DRY RUN TABLE:

Line	score1	score2	print
L1	57		
L2		21	
L3		57	
L4		49	
L5			57:49

CORRECT ANSWER:

57:49

2. Complete the dry run table showing what happens when this code is executed. What is printed?

```
int[] score1 = {57, 57};           // L1
int[] score2 = {21, 21};           // L2
score2 = score1;                   // L3
score2[1] = 27;                    // L4
System.out.println(score1[1] + ":" + score2[1]); // L5
```

In this table @1 and @2 represent the addresses of storage space on the heap. When an array is initialised, the data is stored in the storage space at that address and the address is stored in the named variable as shown. The first line completed shows the result of executing the first line of code.

DRY RUN TABLE:

Line	score1	score2	@1		@2		print
			[0]	[1]	[0]	[1]	
L1	@1		57	57			
L2		@2			21	21	
L3		@1					
L4				27			
L5							27:27

CORRECT ANSWER:

27:27

EXPLANATION:

Because the array values are stored as references, the assignment makes both point to the same place and so when a change to an array element is made it changes it at the place both are pointing to. The effect is therefore different to that of variables holding values that are not objects (e.g. integers).

Overleaf is a dry run sequence as diagrams:

```
int[] score1 = {57, 57};
score1 → {57, 57}
```

```
int[] score2 = {21, 21};
score1 → {57, 57}
score2 → {21, 21}
```

```
score2 = score1;
score1 → {57, 57}
score2 → {57, 57}
```

```
score2[1] = 27;
score1 → {57, 27}
score2 → {57, 27}
```

3. Complete the dry run table showing what happens when this code is executed. What is printed?

```
int[] a = {1,2,3,4};           // L1
int[] b = {1,2,3,4};           // L2
int result = 2;                 // L3
if (a==b)                       // L4
    result = 0;                 // L5
else
    result = 1;                 // L6
System.out.println(result);     // L7
```

DRY RUN TABLE:

L	a	b	@1				@2				result	a==b ?	print
			[0]	[1]	[2]	[3]	[1]	[2]	[3]	[4]			
L1	@1		1	2	3	4							
L2		@2					1	2	3	4			
L3											2		
L4												@1==@2 ? false	
L6											1		
L7													1

CORRECT ANSWER:

1

EXPLANATION:

```
int[] a = {1,2,3,4};
a  $\longrightarrow$  {1, 2, 3, 4}

int[] b = {1,2,3,4};
b  $\longrightarrow$  {1, 2, 3, 4}
```

Arrays behave apparently differently to integers when comparing using ==. As a and b are declared separately, each is given a separate area on the heap to store the actual array values. This means that the bit patterns stored in variables a and b are different as they are references to different places on the heap. The test a == b compares the values in a and b themselves. It does not compare what they are pointing to. That means it compares the references. As those references – ie the bit patterns in a and b - are different, the test is false. The else branch is executed and variable result is therefore set to 1 and 1 is printed.

4. Consider the following method:

```
public static void wombat(int a, int[] b) // L1
{
    a = 1;                                // L2
    b[0] = 2;                             // L3
    return;                               // L4
}
```

Complete the dry run table. What is printed when the following fragment calls the method:

```
int x = 5;                                // L5
int[] y = {6, 7, 8};                     // L6
wombat(x, y);                             // L7
System.out.println(x + ":" + y[0]);      // L8
```

DRY RUN TABLES:

Line	x	y	@1			wombat(x, y)	print
			[0]	[1]	[2]		
L4	5						
L5		@1	6	7	8		
L6			2			wombat(5, @1)	
L7							5:2

wombat(5 , @1)						
Line	a	b	@2			return
			[0]	[1]	[2]	
L1	5	@1				
L2	1					
L3						
L4						void

CORRECT ANSWER:

5:2

EXPLANATION:

x is set to 5 and y to point to the array {6, 7, 8}.

When wombat is called it is passed x and y.

The first step in executing the method is therefore to put a copy of `x` into the newly created local variable `a` and to put a copy of `y` into local variable `b`. As `x` and `a` are integers in their case it means `a` holds the value 5. As `y` and `b` are arrays `b` doesn't get the values in the array but a copy of the pointer to where those values are stored on the stack.

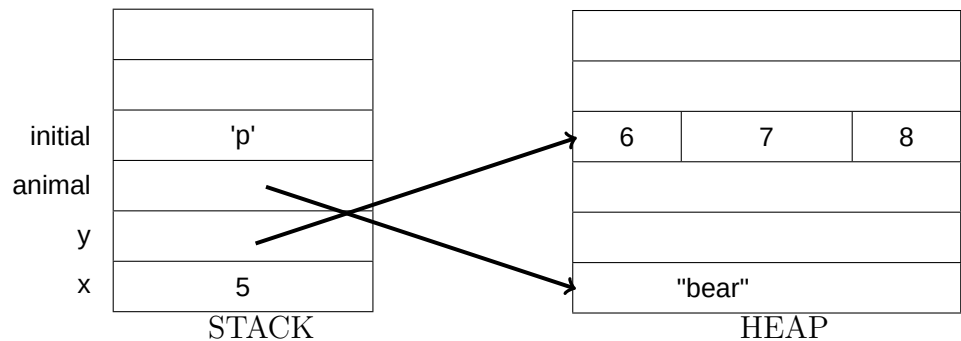
When `a` is changed to 1 in the method it is only variable `a` that is changed. It has no effect at all on `x`. When `b[0]` is assigned to in the method, we follow the reference from `b` and get to the actual data. We go to position 0 which is currently 6. That value is changed to 2. However this is the same place in the heap that `y` is pointing to. When the method returns variables `a` and `b` disappear. Variable `x` has been untouched and still holds 5. Variable `y` itself is also untouched – it still points to the same place as before. However the data in that place on the heap has changed. `y[0]` is actually now holding value 2.

5. Complete the picture showing the way data is stored on the stack and heap after the following sequence of declarations.

```
int x = 5;
int[] y = {6, 7, 8};
String animal = "bear";
char initial = 'p';
```

Draw the String as if fitting in a single cell.

CORRECT ANSWER:



EXPLANATION:

Values of simple types (here the integer and the character) are stored on the stack. Values of compound types (array and String) have a reference stored on the stack that points to an address on the heap.

(NB actual heap position of data depends on where there is free space).